

Nemobot Games: Crafting Strategic AI Gaming Agents for Interactive Learning with Large Language Models

Chee Wei Tan, Yuchen Wang, Shangxin Guo

Abstract—This paper introduces a new paradigm for AI game programming, leveraging large language models (LLMs) to extend and operationalize Claude Shannon’s taxonomy of game-playing machines. Central to this paradigm is *Nemobot*, an interactive agentic engineering environment that enables users to create, customize, and deploy LLM-powered game agents while actively engaging with AI-driven strategies. The LLM-based chatbot, integrated within Nemobot, demonstrates its capabilities across four distinct classes of games. For *dictionary-based games*, it compresses state-action mappings into efficient, generalized models for rapid adaptability. In *rigorously solvable games*, it employs mathematical reasoning to compute optimal strategies and generates human-readable explanations for its decisions. For *heuristic-based games*, it synthesizes strategies by combining insights from classical minimax algorithms (see, e.g., [1]) with crowd-sourced data. Finally, in *learning-based games*, it utilizes reinforcement learning with human feedback and self-critique to iteratively refine strategies through trial-and-error and imitation learning. *Nemobot* amplifies this framework by offering a programmable environment where users can experiment with tool-augmented generation and fine-tuning of strategic game agents. From strategic games to role-playing games, Nemobot demonstrates how AI agents can achieve a form of self-programming by integrating crowdsourced learning and human creativity to iteratively refine their own logic. This represents a step toward the long-term goal of self-programming AI.

Index Terms—Game learning machines, Large language model, AI games, Self-programming AI, Reinforcement learning.

I. INTRODUCTION

CLAUDE Shannon, a pioneer in artificial intelligence, posed a thought-provoking question in his seminal work [2]: “Can we program a digital computer so that (eventually) 99 percent of the orders it follows are written by the computer itself, rather than the few percent in current programs?” This question encapsulates the vision of self-programming systems, a concept that remains a core challenge in AI research. Similarly, Marvin Minsky argued in [3] that programming’s structured rules are not limitations but tools for refining and clarifying poorly understood ideas, akin to how artists and composers use formal constraints to express creativity. Shannon’s vision of computers autonomously generating their own instructions has become increasingly relevant in agentic engineering, where large language models (LLMs) now produce vast amounts of code, driving self-evolving AI systems and interactive applications such as games.

C. W. Tan and Y. Wang are with Nanyang Technological University, Singapore, Nanyang Ave., Singapore (e-mail: cheewei.tan@ntu.edu.sg).
S. Guo is with Nautilus Software Technologies Limited, Hong Kong.

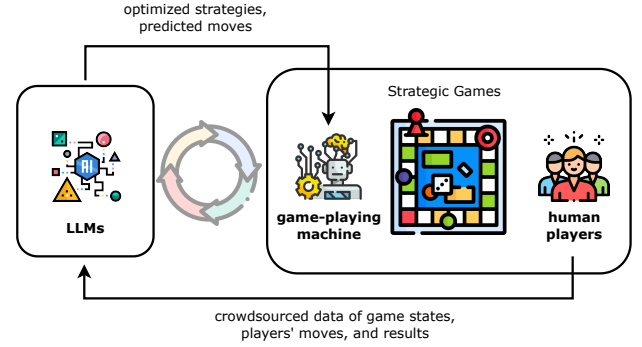


Fig. 1: Crowdsourcing and strategy optimization in game-playing AI. The LLM generates optimized strategies for game-playing agents, while game states and results from interactions with human players are fed back to train the LLM, creating a self-reinforcing cycle of improvement through crowdsourcing.

Recent surveys, such as [4], [5], explore advances in strategic AI game agents to enhance adaptability and reasoning using LLMs. Research into specific applications such as combining language models with strategic reasoning in the game of Diplomacy [6] and using LLMs as testers to measure game difficulty [7] demonstrates the versatility of LLMs in imperfect-information games. Approaches like ChessGPT [8] and AlphaStar [9] bridge policy learning with language modeling, emphasizing the importance of synergizing reinforcement learning and natural language processing. The development of systems such as Suspicion-Agent [10], which integrates theory-of-mind-aware reasoning with OpenAI’s GPT-4, further extends the horizon of multi-agent interactions in imperfect information settings.

Game boards serve as microworlds: discrete, observable world states that provide a structured, controllable testbed for studying *representation learning of world models* in LLMs. When trained on game transcripts and rules, LLMs can learn implicit representations that encode not only legal moves but also temporal dependencies, hidden states, and probabilistic opponent strategies. For instance, [11] demonstrate that chess gameplay enables LLMs to track board states and predict moves, providing evidence that strategic environments can induce meaningful internal representations of game dynamics. This aligns with findings by [12], who show that transformers trained on synthetic sequential prediction tasks spontaneously

the potential and limitations of collective decision-making in human-machine collaboration. Crowdsourcing offers a scalable way to improve natural language understanding by leveraging diverse participant perspectives to refine responses [40], [45]. Crowdsourcing will matter in agentic engineering by providing scalable human oversight and diverse training data that autonomous agents cannot generate on their own.

In strategic game programming, crowdsourcing enhances LLM fine-tuning by improving reinforcement learning models and simplifying evaluation. It generates prompts for strategy, move explanations, opponent simulations, and diverse gameplay styles, boosting AI adaptability. This collaborative process aggregates comprehensive datasets, uncovers high-performing patterns, and fosters innovation. By integrating these insights into LLM agents (e.g., Nemobot in Section III), it scales human creativity, enhances human-AI augmentation, and drives new programming paradigms in strategic AI game development, as illustrated in Figure 1.

The learning process in complex AI game playing will involve significant trial and error, but crowdsourcing can reduce the overhead by aggregating diverse feedback and human insights. Programmable prompts amplify crowdsourcing by providing a structured framework to encode, refine, and evaluate collective human expertise. Through LLM functions, these prompts enable agents to dynamically simulate strategic decision-making and problem-solving. Crowdsourcing from a diverse community exposes LLM agents to a wide range of reasoning styles and creative strategies, aligning with Michie’s *neuralized memoization*. This approach allows for iterative improvement, combining learned strategies with programmable modules to enhance performance over time.

D. AI Game Programming Agent

To leverage the full potential of LLM capabilities, a tool must be developed with the following characteristics:

- *Optimize both computational efficiency and strategic effectiveness* by incorporating chain-of-thought reasoning, allowing developers to explore the inner workings and decision-making processes of AI agents. This fosters a deeper understanding of how strategies are generated and refined, as LLMs can explain the logic behind optimal moves and provide insights into trial-and-error learning.
- *Facilitates collaborative and scalable development* by utilizing LLM functions to generate code and assist in debugging, enabling developers to collaborate on strategy formulation and game design while supporting various approaches. This scalability also encourages team-based improvements and iterative strategy refinement.
- *Provides accessibility for developers with varying expertise* by offering an adaptable framework for hands-on exploration, with LLM-powered tools that simplify experimentation. The tool’s tiered classification, based on Shannon’s taxonomy, coupled with LLM-driven code generation, caters to both novice and advanced programmers. By facilitating dynamic collaboration with LLMs, the system enables efficient real-time generation, optimization, and adaptation of strategies. Ultimately,

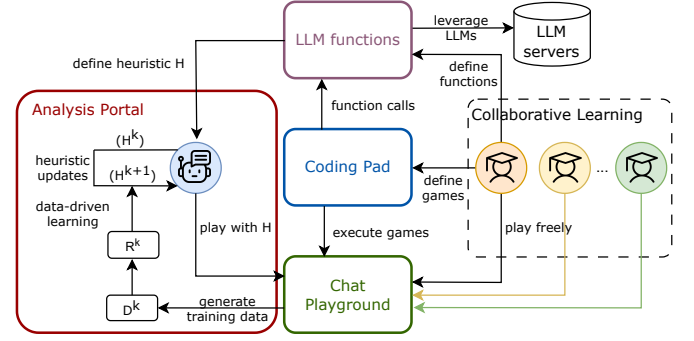


Fig. 3: System Flow of Nemobot. Students program strategic games, build AI with given heuristics and conduct data-driven AI training.

the goal is to achieve scalable neuralized memoization, enhancing algorithmic efficiency and empowering developers to innovate and refine solutions seamlessly.

III. NEMOBOT GAME PROGRAMMING

We present Nemobot, a programming framework for creating, training, and testing AI game agents on platforms like Facebook Messenger and Telegram. In this section, we provide an architectural overview of the system, detail the interactive training workflow, and describe the tiered curriculum designed to support developers of varying expertise.

A. Overview of Nemobot

Nemobot provides a dynamic and intuitive environment where developers can design and train AI agents for strategic games, gaining insights into the underlying mechanisms of AI. As illustrated in Figure 3, the platform comprises a **coding pad** for programming both the game logic and the AI’s reasoning heuristics, a **chat playground** for interacting with the AI to generate and test data, and an **analysis portal** for evaluating AI performance and refining heuristics.

The process begins with users designing a two-player strategic game in which players alternate turns, taking actions and accruing rewards until the game concludes. A NodeJS-based template is provided to streamline this process, featuring customizable functions to define the initial state S_0 , the transition function $f(S_{i+1}|S_i, a_i)$ —where a_i represents the player action at state S_i —and the termination state S_e . Users then program their AI agent by defining a **heuristic** $H^0(a_i|S_i)$, which dictates the agent’s decision-making in various scenarios.

Nemobot also empowers users to incorporate advanced algorithm design by leveraging LLM functions. Users can bypass complex programming challenges by specifying high-level logic in natural language such as descriptions and examples within the platform. These LLM functions, as shown in Figure 4 part (c), are seamlessly integrated into the main game flow, leveraging LLMs’ natural language understanding and code generation capabilities. This feature ensures accessibility for developers with diverse programming backgrounds while maintaining a robust programming paradigm for AI game development.

TABLE I: Nemobot games, powered by OpenAI Codex and Application Programming Interface (API) between 2021 and 2023, categorized according to Shannon’s game taxonomy. These games demonstrate a versatile range of game programs. The commands can be accessed on Facebook Messenger.

Game Description	LLM Agent Enhancement	Category	Link
To run the Tic Tac Toe game, type the command: tic_tac_toe hkalexling.	LLM function to query moves	1	Nemobot Game
To run the classic Nim game, type the command: lollipops hkalexling.	LLM function to query moves and optimal strategy algorithm by code generation	2	Nemobot Game
To run the Mancala game, type the command: temp awari_server.	LLM function to query moves and collect human-AI gameplay data in a leaderboard	3	Nemobot Game
Physics learning quiz game challenges players on Newton’s three laws.	Offers contextual explanations, personalized feedback, hints, and motivational dialogue.	1	Newton’s Laws
Single-player RPG where players master math topics by selecting challenges.	Using crowdsourcing via LLM functions, it offers personalized explanations and answers questions.	3	Math Video Recommender
Audio recommendation system game leveraging crowdsourced audio data.	LLM functions enhance the game by interpreting user feedback and suggesting improvements.	4	Audio Tagging
Players solve coding challenges in an IDE and provide peer feedback.	It provides hints, analyzes solutions, and simulates expert coaching.	4	Code Mentor
Role-playing game with dynamic dialogue trees.	LLM functions analyze player inputs and generate adaptive dialogue trees.	4	Role Playing Game

To address the limitations, we employ the LLM gaming agent, which replicates the behavior of a dictionary-based system without the need to explicitly store an extensive collection of state-action pairs or create complex nested conditional loops. The user interacts with Nemobot by making moves, and the LLM responds as the opposing player, maintaining the deterministic characteristics of a dictionary-based system. To facilitate interaction, the 3×3 game board is represented with cells labeled numerically from 0 to 8, corresponding to the positions on the board. Users input their moves by specifying the cell where they want to place their mark (e.g., “0” for the top-left corner or “4” for the center). To communicate with the LLM agent, the game state is serialized into a structured textual format that captures the positions of X, O, and empty cells. This structured input effectively describes the current state of the game in a manner that the agent can process and respond to. For instance, assuming the LLM agent is already aware of its role and the rules embedded within the configuration of Nemobot, the prompt for a given state could be written as below.

Example Prompt: Tic Tac Toe Strategy

Current Configuration: Cell 0 (Top-Left) is occupied by ‘X’. Cell 4 (Center) is occupied by ‘O’. All remaining cells are empty.

Objective: Analyze the board state and execute the optimal move for Player ‘O’ to prevent a loss or secure a win.

Instead of precomputing all possible moves and storing them in a dictionary, the LLM gaming agent uses its pre-trained knowledge and contextual understanding to generate optimal moves dynamically. The backend sends this query to the LLM through an API, and the LLM responds with the most strategic move based on the current configuration. This

dynamic querying process eliminates the need for exhaustive storage, effectively compressing the “dictionary” into the inference capabilities of the LLM gaming agent. For any given game state, Nemobot ensures that the responses of the agent are consistent and deterministic, adhering to the foundational principles of traditional dictionary-based machines. Although the LLM gaming agent dynamically generates responses, these responses align with established optimal strategies for Tic Tac Toe, with its decision-making grounded in its pretrained knowledge. This design ensures that the foundational knowledge required for selecting optimal moves is embedded into the LLM once, allowing it to infer the best moves even for previously unseen positions. This approach significantly reduces memory usage and computational overhead compared to traditional methods that rely on exhaustive searches or evaluating nested conditional loops for each decision. By leveraging the ability of the LLM gaming agent to generalize and interpret game rules, Nemobot could achieve enhanced scalability and adaptability, delivering efficient and accurate gameplay without the need for repeated rule-based evaluations.

1) *Machines using Rigorous Mathematical Formulas in Nim Game:* Nim is a classic combinatorial game and an example of Shannon’s second category of game-playing machines, which use mathematical formulas. Its optimal strategies are based on the Nim-sum, determining winning or losing positions. In Nemobot, the player forced to take the last object loses. The AI calculates the Nim-sum at each turn, using the formula to determine the best move: a nonzero Nim-sum indicates a winning position, while a zero Nim-sum signals a losing position. The LLM translates the Nim-sum calculations into natural language, offering insights into the optimal moves and making the game more educational and accessible to users.

The game logic serializes the current state of the board into a structured text format, detailing the number of stones remaining and whose turn it is. The serialized state is then